

Q1. Explain Reactive Risk Strategy and Proactive Risk Strategy with suitable examples.

Answer

Introduction

Risk management is an important activity in software engineering. During software development, many uncertainties may arise such as:

- Requirement changes
- Staff shortage
- Technology failure
- Schedule delays

To handle these uncertainties, organizations adopt risk management strategies.

Two important strategies are:

1. Reactive Risk Strategy
2. Proactive Risk Strategy

According to the Software Engineering Module-IV notes, reactive and proactive strategies are fundamental approaches used in software risk management.

What is Risk?

A software risk is a potential problem that may negatively affect the success of a software project.

Risk always involves:

1. Uncertainty
2. Loss

If the risk becomes real, unwanted consequences occur.

Reactive Risk Strategy

Definition

Reactive Risk Strategy is an approach in which the software team takes action only after a problem occurs.

This strategy does not focus on preventing risks in advance.

The team reacts after the issue becomes serious.

Working of Reactive Strategy

1. Project proceeds normally
 2. Problem suddenly occurs
 3. Team enters “fire-fighting mode”
 4. Immediate corrective actions taken
 5. Crisis management performed
-

Characteristics

1. No advance planning
 2. Action taken after risk occurrence
 3. Crisis-oriented approach
 4. Higher possibility of project delay
 5. Temporary solutions are common
-

Diagram

Project Starts

↓

Risk Occurs

↓

Fire Fighting Mode

↓

Problem Solved

Example

Suppose a software company is developing an Online Banking System.

During development:

- Main database server crashes unexpectedly.

Since no backup strategy was prepared:

- Team starts emergency recovery process.
- Project schedule gets delayed.

This is a Reactive Risk Strategy because action is taken only after failure occurs.

Advantages of Reactive Strategy

1. Simple to implement
 2. Requires less initial planning
 3. Suitable for small projects
 4. Lower initial cost
-

Disadvantages of Reactive Strategy

1. High project risk
 2. Increased development cost
 3. Delays in project delivery
 4. Poor resource utilization
 5. Stressful working environment
-

Proactive Risk Strategy

Definition

Proactive Risk Strategy is an approach in which risks are identified, analyzed, and managed before they occur.

This strategy focuses on prevention and preparedness.

Working of Proactive Strategy

1. Risks are identified early
 2. Probability and impact analyzed
 3. Risks ranked according to importance
 4. Mitigation plans prepared
 5. Continuous monitoring performed
-

Characteristics

1. Preventive approach
2. Proper planning
3. Risk analysis performed
4. Backup plans available

5. Continuous monitoring

Diagram

Risk Identification



Risk Analysis



Risk Mitigation Plan



Risk Monitoring

Example

Suppose a company is developing a Hospital Management System.

Possible risk:

- Key developer may leave project.

To handle this risk:

- Backup developers are trained.
- Documentation maintained properly.

If developer leaves:

- Another developer continues work smoothly.

This is Proactive Risk Strategy because preparation was done before risk occurred.

Advantages of Proactive Strategy

1. Reduces project failure chances
 2. Better planning and control
 3. Improves project quality
 4. Reduces development cost
 5. Increases customer satisfaction
-

Disadvantages of Proactive Strategy

1. Requires more planning effort

2. Higher initial cost
3. Time-consuming analysis

Difference Between Reactive and Proactive Risk Strategy

Reactive Strategy	Proactive Strategy
Action taken after problem occurs	Action taken before problem occurs
Crisis management approach	Prevention approach
No advance planning	Proper planning performed
Higher project risk	Lower project risk
Fire-fighting mode	Risk mitigation planning

Real Life Applications

Strategy Example

Reactive Fixing server crash after failure

Proactive Maintaining backup servers

Conclusion

Reactive Risk Strategy handles problems after occurrence, while Proactive Risk Strategy identifies and prevents risks in advance. Modern software organizations prefer proactive strategies because they improve project stability, quality, and success rate.

Q2. Explain Software Risks and their types with examples.

Answer

Introduction

Software development projects involve uncertainties that may affect:

- Cost
- Schedule
- Quality
- Performance

These uncertainties are called software risks.

Effective risk management helps organizations reduce project failures.

Definition of Software Risk

Software Risk is the possibility of suffering loss due to uncertain events during software development.

A risk may or may not occur, but if it occurs, it negatively affects the project.

According to the module notes:

- Risk involves uncertainty and loss.
-

Characteristics of Software Risk

1. Uncertainty
 2. Potential loss
 3. Impacts project objectives
 4. Can affect quality and schedule
 5. Requires proper management
-

Types of Software Risks

According to software engineering, major risks are:

1. Project Risk
 2. Technical Risk
 3. Business Risk
 4. Known Risks
 5. Predictable Risks
 6. Unpredictable Risks
-

1. Project Risk

Definition

Project risks affect:

- Project schedule
- Budget
- Resources

Causes

1. Poor planning
2. Staff shortage
3. Unrealistic deadlines
4. Resource unavailability

Example

A project requiring 10 developers has only 5 available developers.

Result:

- Project delivery delayed.

Effects

1. Schedule delay
2. Increased cost
3. Reduced productivity

2. Technical Risk

Definition

Technical risks affect:

- Software quality
- Technical performance

Causes

1. New technology usage
2. Complex design
3. Integration problems
4. Performance issues

Example

A company uses a new AI framework that developers do not fully understand.

Result:

- Software crashes frequently.
-

Effects

1. Poor software quality
 2. Increased defects
 3. Performance failure
-

3. Business Risk

Definition

Business risks affect:

- Business success
 - Profitability
 - Market acceptance
-

Causes

1. Market competition
 2. Customer rejection
 3. Budget issues
 4. Management changes
-

Example

An e-commerce application fails because customers prefer competitor applications.

Effects

1. Financial loss
 2. Business failure
 3. Reduced customer trust
-

4. Known Risks

Definition

Risks identified during project planning.

Example

- Requirement changes
 - Staff turnover
-

Advantage

Easy to manage because already identified.

5. Predictable Risks

Definition

Risks predicted from previous project experience.

Example

Testing delays in large projects.

6. Unpredictable Risks

Definition

Risks that cannot be predicted easily.

Example

1. Natural disasters
 2. Sudden server failure
 3. Cyber attacks
-

Risk Components and Drivers

Important risk drivers include:

1. Performance risk
2. Cost risk
3. Support risk
4. Schedule risk

Risk Management Process

Risk Identification



Risk Analysis



Risk Mitigation



Risk Monitoring

Importance of Risk Management

1. Reduces project failure
 2. Improves planning
 3. Better quality software
 4. Reduces cost overruns
 5. Improves customer satisfaction
-

Conclusion

Software risks are unavoidable in software projects. Proper identification and management of project, technical, and business risks help organizations successfully complete projects within cost, schedule, and quality constraints.

Q3. Describe the Risk Identification Process and explain the Risk Item Checklist.

Answer

Introduction

Risk identification is one of the most important steps in software risk management.

If risks are not identified early:

- Projects may fail
- Costs may increase
- Deadlines may be missed

Therefore, software engineers must identify possible risks before development progresses.

Definition of Risk Identification

Risk Identification is the process of recognizing, analyzing, and documenting possible risks that may affect the software project.

The goal is:

- To discover potential problems early.
-

Objectives of Risk Identification

1. Detect possible risks early
 2. Reduce uncertainty
 3. Improve project planning
 4. Prevent project failure
 5. Prepare mitigation strategies
-

Types of Risks Identified

According to the module notes:

1. Generic Risks
 2. Product-Specific Risks
-

1. Generic Risks

Risks common to all software projects.

Examples

1. Staff shortage
 2. Budget constraints
 3. Schedule delays
-

2. Product-Specific Risks

Risks specific to particular software products.

Examples

1. Complex algorithms

2. Security vulnerabilities
 3. Hardware compatibility issues
-

Risk Identification Process

The process includes several systematic steps.

Step 1 — Understand the Project

Study:

- Requirements
 - Objectives
 - Constraints
-

Step 2 — Identify Possible Risks

Discuss with:

- Developers
 - Customers
 - Managers
 - Testers
-

Step 3 — Categorize Risks

Classify risks into:

- Technical
 - Project
 - Business
-

Step 4 — Document Risks

Prepare risk documentation.

Step 5 — Prioritize Risks

Rank risks according to:

- Probability

- Impact

Diagram of Risk Identification Process

Project Study



Risk Identification



Risk Classification



Risk Documentation



Risk Prioritization

Risk Item Checklist

Definition

Risk Item Checklist is a structured list of important factors used to identify software risks systematically.

It helps software teams avoid missing important risks.

Risk Item Checklist Factors

According to the module notes, the checklist includes:

1. Product Size
 2. Business Impact
 3. Customer Characteristics
 4. Process Definition
 5. Development Environment
 6. Technology to be Built
 7. Staff Size and Experience
-

1. Product Size

Large projects generally involve:

- More complexity
 - Higher risk
-

Example

Banking software with thousands of modules.

2. Business Impact

Analyzes effect of project on business.

Example

Failure of online payment system causing financial loss.

3. Customer Characteristics

Considers:

- Customer involvement
 - Requirement clarity
-

Example

Frequent requirement changes by customer.

4. Process Definition

Checks whether software process is clearly defined.

Example

Lack of testing procedures increases risk.

5. Development Environment

Includes:

- Hardware
- Software tools
- Infrastructure

Example

Outdated development tools causing delays.

6. Technology to be Built

New or complex technologies increase risk.

Example

Using blockchain technology without experienced developers.

7. Staff Size and Experience

Experienced staff reduces risk.

Example

Inexperienced developers causing coding errors.

Advantages of Risk Item Checklist

1. Systematic risk identification
 2. Reduces missing risks
 3. Improves project planning
 4. Better project monitoring
 5. Improves software quality
-

Conclusion

Risk identification is a critical activity in software engineering. Using a Risk Item Checklist helps organizations systematically identify and manage risks, improving project success and software quality.

Q4. Explain Risk Projection and the steps involved in developing a Risk Table.**Answer****Introduction**

After identifying risks, software engineers must analyze:

- Probability of occurrence
- Impact on project

This process is called Risk Projection.

Risk projection helps organizations prioritize important risks and prepare mitigation strategies.

Definition of Risk Projection

Risk Projection is the process of estimating the likelihood and consequences of identified risks.

According to the module notes:

- Risk projection evaluates probability and consequences of risks.
-

Objectives of Risk Projection

1. Estimate risk probability
 2. Analyze risk impact
 3. Prioritize risks
 4. Improve decision-making
 5. Prepare mitigation plans
-

Risk Projection Parameters

Two important parameters are analyzed:

1. Probability of Risk
 2. Impact of Risk
-

1. Probability

Measures likelihood of risk occurrence.

Example

High Probability → 80%

Medium Probability → 50%

Low Probability → 20%

2. Impact

Measures seriousness of risk effect.

Example

- Minor
 - Critical
 - Catastrophic
-

Four Steps of Risk Projection

According to the module notes, four major steps are involved.

Step 1 — Establish Risk Scale

Define probability levels.

Example

Probability Meaning

80%	High
50%	Medium
20%	Low

Step 2 — Determine Consequences

Analyze consequences if risk occurs.

Example

If server fails:

- Entire system becomes unavailable.
-

Step 3 — Estimate Impact

Estimate effect on:

- Cost
- Schedule

- Performance
 - Quality
-

Example

Requirement changes may delay project by 2 months.

Step 4 — Assess Accuracy

Ensure:

- Estimates are realistic
 - No misunderstandings exist
-

Risk Table

Definition

A Risk Table is a structured table used to rank risks according to probability and impact.

It helps:

- Prioritize critical risks.
-

Steps in Developing Risk Table

Step 1 — Identify Risks

List all possible risks.

Step 2 — Categorize Risks

Categories:

- Technical
 - Project
 - Business
-

Step 3 — Assign Probability

Estimate occurrence probability.

Step 4 — Assign Impact

Estimate seriousness of effect.

Step 5 — Rank Risks

Arrange according to severity.

Example Risk Table

Risk	Probability	Impact	Priority
Staff shortage	70%	High	High
Requirement changes	60%	Medium	Medium
Hardware failure	30%	Low	Low

Risk Impact Categories

According to module notes:

1. Catastrophic
 2. Critical
 3. Marginal
 4. Negligible
-

Diagram of Risk Projection

Risk Identification

↓

Probability Estimation

↓

Impact Analysis

↓

Risk Table Creation

Advantages of Risk Projection

1. Better planning

2. Prioritized risk handling
 3. Reduced project failure
 4. Improved project control
 5. Better resource allocation
-

Conclusion

Risk projection helps software organizations analyze and prioritize risks systematically. Developing a Risk Table improves decision-making and enables effective risk management throughout the software development process.

Q5. Explain Risk Refinement using CTC format with an example.

Answer

Introduction

After risks are identified and projected, they must be analyzed in greater detail. This detailed analysis process is called:

- Risk Refinement

Risk refinement helps software engineers understand:

- Root causes
- Conditions
- Consequences

This enables better mitigation planning.

Definition of Risk Refinement

Risk Refinement is the process of describing and analyzing a software risk in greater detail.

According to the module notes:

- Risk refinement specifies risk using CTC format.
-

Purpose of Risk Refinement

1. Understand risks clearly
2. Identify root causes
3. Improve mitigation planning
4. Analyze consequences

5. Reduce uncertainty

CTC Format

Risk refinement is represented using:

- CTC Format

CTC stands for:

1. Condition
 2. Transition
 3. Consequence
-

1. Condition

Represents circumstances causing risk.

Example

Key developer may leave project.

2. Transition

Represents change from normal state to risk state.

Example

Developer resigns during critical development phase.

3. Consequence

Represents effect of risk occurrence.

Example

Project delayed by 2 months.

Diagram of CTC Format

Condition

↓

Transition



Consequence

Working of Risk Refinement

1. Identify main risk
 2. Identify sub-conditions
 3. Analyze transitions
 4. Determine consequences
 5. Prepare mitigation plan
-

Example of Risk Refinement

Risk

“Inexperienced developers in AI project”

CTC Analysis

Condition

Developers do not have AI experience.

Transition

Complex AI modules become difficult to implement.

Consequence

1. Increased defects
 2. Delayed project delivery
 3. Reduced software quality
-

Another Example

Risk

“Requirement changes by customer”

Condition

Customer requirements are unclear.

Transition

Frequent changes requested during development.

Consequence

1. Increased development cost
 2. Schedule delays
 3. Rework required
-

Advantages of Risk Refinement

1. Better understanding of risks
 2. Helps identify root causes
 3. Improves mitigation planning
 4. Better project monitoring
 5. Reduces project uncertainty
-

Importance in Software Engineering

Risk refinement helps:

- Project managers
- Developers
- Testers

understand detailed risk behavior and prepare effective strategies.

Applications

1. Banking software
2. Healthcare systems
3. E-commerce applications
4. Cloud systems

5. AI applications

Conclusion

Risk Refinement using the CTC format provides a detailed understanding of software risks by analyzing conditions, transitions, and consequences. It helps organizations develop effective mitigation strategies and improve software project success.

Q6. Explain Risk Mitigation, Risk Monitoring and Risk Management.

Answer

Introduction

Software projects are exposed to many risks such as:

- Requirement changes
- Technical failures
- Staff turnover
- Budget issues

After identifying and analyzing risks, software engineers must take actions to:

1. Reduce risks
2. Monitor risks
3. Handle risks if they occur

These activities are called:

- Risk Mitigation
- Risk Monitoring
- Risk Management

According to the module notes, these are major activities in software risk management.

1. Risk Mitigation

Definition

Risk Mitigation is the process of avoiding risks or reducing the probability and impact of risks before they occur.

The goal is:

- Prevention of problems.
-

Objectives of Risk Mitigation

1. Reduce risk occurrence
 2. Minimize project losses
 3. Improve project stability
 4. Ensure smooth development
 5. Improve software quality
-

Steps for Risk Mitigation

According to module notes, mitigation steps include:

1. Communicate with staff to identify risks
 2. Eliminate causes before project starts
 3. Develop backup policies
 4. Maintain documentation properly
 5. Conduct peer reviews
 6. Assign backup staff
-

Example

Suppose risk:

- Key developer may leave project.

Mitigation steps:

- Train backup developer
- Maintain proper documentation

Result:

- Work continues smoothly even if developer leaves.
-

Advantages of Risk Mitigation

1. Prevents project failure
 2. Reduces development cost
 3. Improves quality
 4. Reduces delays
 5. Increases customer satisfaction
-

2. Risk Monitoring

Definition

Risk Monitoring is the continuous process of tracking identified risks and checking whether mitigation plans are working properly.

Objectives of Risk Monitoring

According to module notes:

1. Check whether risks occur
 2. Ensure mitigation steps are applied
 3. Collect information for further analysis
-

Activities in Risk Monitoring

1. Track identified risks
 2. Monitor warning signs
 3. Review mitigation progress
 4. Update risk status
 5. Analyze new risks
-

Example

Suppose risk:

- Server overload

Monitoring activities:

- Daily server performance tracking
 - CPU usage analysis
 - Alert systems
-

Diagram

Risk Identified

↓

Risk Monitored

↓

Warning Signs Detected



Corrective Action Taken

Advantages of Risk Monitoring

1. Early problem detection
 2. Better control
 3. Reduced uncertainty
 4. Faster corrective action
 5. Improved project management
-

3. Risk Management

Definition

Risk Management is the complete process of identifying, analyzing, mitigating, monitoring, and controlling risks throughout the software project.

Purpose

To minimize negative effects of risks on:

- Cost
 - Schedule
 - Quality
 - Performance
-

Activities in Risk Management

1. Risk identification
 2. Risk analysis
 3. Risk prioritization
 4. Risk mitigation
 5. Risk monitoring
 6. Contingency planning
-

Contingency Planning

According to module notes:

- Backup plans should always be ready.
-

Example

Suppose:

- Cloud server fails

Contingency plan:

- Switch to backup server immediately.
-

Risk Management Process

Risk Identification

↓

Risk Analysis

↓

Risk Mitigation

↓

Risk Monitoring

↓

Contingency Planning

Importance of Risk Management

1. Reduces project failure
 2. Improves software quality
 3. Better project control
 4. Reduces financial loss
 5. Improves customer confidence
-

Real Life Example

In Online Banking System:

- Backup servers

- Security monitoring
- Disaster recovery plans

are examples of risk management activities.

Difference Between Mitigation, Monitoring and Management

Activity	Purpose
Risk Mitigation	Prevent risk occurrence
Risk Monitoring	Track and observe risks
Risk Management	Overall risk handling process

Conclusion

Risk Mitigation, Risk Monitoring, and Risk Management are essential activities in software engineering. They help organizations reduce uncertainties, prevent failures, and improve project success and software quality.

Q7. What is an RMMM Plan? Explain its importance in software projects.

Answer

Introduction

Software projects involve many risks such as:

- Technical problems
- Requirement changes
- Resource shortages
- Schedule delays

To handle these risks systematically, organizations prepare:

- RMMM Plan

RMMM stands for:

1. Risk Mitigation
2. Risk Monitoring
3. Risk Management

According to the module notes, RMMM plan is an important document used in software project management.

Definition of RMMM Plan

RMMM Plan is a formal document that describes risk analysis, mitigation strategies, monitoring activities, and management procedures for software project risks.

It acts as:

- A roadmap for handling risks.
-

Objectives of RMMM Plan

1. Identify project risks
 2. Reduce risk probability
 3. Monitor risks continuously
 4. Prepare contingency plans
 5. Improve project success rate
-

Components of RMMM Plan

1. Risk Mitigation

Defines:

- Steps to avoid or reduce risks
-

Example

Train backup developers to handle staff turnover.

2. Risk Monitoring

Defines:

- How risks will be tracked
-

Example

Weekly project review meetings.

3. Risk Management

Defines:

- Actions taken if risk occurs
-

Example

Using backup servers during server failure.

Structure of RMMM Plan

Risk Identification

↓

Risk Mitigation Plan

↓

Risk Monitoring Plan

↓

Risk Management Plan

Contents of RMMM Plan

1. Risk description
 2. Risk probability
 3. Risk impact
 4. Mitigation steps
 5. Monitoring activities
 6. Contingency plans
 7. Responsible persons
-

Risk Information Sheet (RIS)

According to module notes:

- Individual risks may be documented using RIS (Risk Information Sheet).
-

Example of RMMM Plan

Risk	Mitigation	Monitoring	Management
Staff shortage	Train backups	Weekly reviews	Hire temporary staff
Server crash	Maintain backups	Monitor servers	Use disaster recovery

Importance of RMMM Plan

1. Reduces Project Failure

Early planning minimizes risk impact.

2. Improves Decision Making

Managers can take better corrective actions.

3. Better Resource Utilization

Resources are planned efficiently.

4. Improves Project Stability

Continuous monitoring keeps project under control.

5. Enhances Customer Satisfaction

Projects are completed on time with better quality.

Advantages of RMMM Plan

1. Better project planning
 2. Reduced uncertainties
 3. Improved software quality
 4. Faster recovery from failures
 5. Improved team coordination
-

Disadvantages

1. Requires additional effort

2. Time-consuming documentation
 3. Difficult for small projects
-

Applications

1. Banking software
 2. Healthcare systems
 3. E-commerce platforms
 4. Cloud applications
 5. Government projects
-

Conclusion

RMMM Plan is an essential part of software project management. It helps organizations identify, monitor, and manage risks systematically, improving project success and reducing failures.

Q8. Explain Software Configuration Management (SCM) and its features.

Answer

Introduction

Software projects continuously undergo changes due to:

- Requirement updates
- Bug fixes
- New features
- Technology improvements

Managing these changes manually becomes difficult.

To control software changes systematically, Software Configuration Management (SCM) is used.

According to module notes, SCM helps identify, organize, and control software modifications.

Definition of SCM

Software Configuration Management (SCM) is the process of identifying, organizing, controlling, and tracking changes made to software during its development life cycle.

Objectives of SCM

1. Control software changes
 2. Maintain software consistency
 3. Manage multiple versions
 4. Improve coordination
 5. Ensure software quality
-

Need for SCM

Without SCM:

- Files may be overwritten
- Wrong versions may be released
- Changes may become untraceable

SCM solves these problems.

Elements of SCM System

According to module notes, SCM system contains:

1. Component Elements
 2. Process Elements
 3. Construction Elements
 4. Human Elements
-

1. Component Elements

Tools used for support activities.

Example

Version control software.

2. Process Elements

Defines actions and tasks.

Example

Change approval procedures.

3. Construction Elements

Tools used for development.

Example

Compilers and build tools.

4. Human Elements

People involved in SCM activities.

Example

Developers and project managers.

Software Configuration Items (SCIs)

SCIs are software items managed under SCM.

According to module notes, examples include:

1. Source code
 2. Documents
 3. User manuals
 4. Test plans
 5. Data files
-

Features of SCM

According to module notes, major SCM features are:

1. Versioning
 2. Dependency Tracking and Change Management
 3. Requirements Tracing
 4. Configuration Management
 5. Audit Trails
-

1. Versioning

Stores multiple software versions.

Example

Version 1.0, Version 2.0

Advantages

- Easy rollback
 - Better release management
-

2. Dependency Tracking and Change Management

Tracks relationships between software components.

Example

Changing database schema may affect application modules.

3. Requirements Tracing

Tracks requirements through development stages.

Example

Requirement linked to design and testing modules.

4. Configuration Management

Maintains project baselines and configurations.

Example

Stable release versions.

5. Audit Trails

Maintains history of:

- Who changed what
- When changes occurred

Example

Git commit history.

SCM Process Diagram

Identification

↓

Version Control

↓

Change Control

↓

Configuration Audit

↓

Status Reporting

Advantages of SCM

1. Better version control
 2. Improved team coordination
 3. Easy change tracking
 4. Better software quality
 5. Faster recovery from errors
-

Applications

1. Web applications
 2. Banking systems
 3. ERP software
 4. Mobile applications
 5. Cloud platforms
-

Conclusion

Software Configuration Management is an essential activity in software engineering that helps manage software changes effectively. SCM improves version management, quality, maintainability, and team collaboration.

Q9. Describe the SCM Repository and its functions.

Answer

Introduction

Software Configuration Management (SCM) requires a central storage area where all software artifacts can be managed efficiently.

This storage mechanism is called:

- SCM Repository

The repository stores and controls:

- Source code
- Documents
- Test cases
- Configurations
- Change records

According to module notes, SCM repository is a set of mechanisms and data structures used to manage software changes effectively.

Definition of SCM Repository

SCM Repository is a centralized storage system used to maintain software configuration items and manage software changes systematically.

Purpose of SCM Repository

1. Store project artifacts
 2. Manage software versions
 3. Support team collaboration
 4. Track software changes
 5. Improve software consistency
-

Contents of SCM Repository

According to module notes, repository may contain:

1. Business content
 2. Project management content
 3. Model content
 4. Construction content
 5. Validation & Verification content
 6. Documents
-

Examples of Stored Items

1. Source code
 2. Requirement documents
 3. Test cases
 4. Design diagrams
 5. User manuals
 6. Configuration files
-

SCM Repository Structure

SCM Repository

↓

| Source Code |

| Documents |

| Test Cases |

| Change Requests |

| Build Files |

Functions of SCM Repository

According to module notes, SCM repository performs several important functions.

1. Data Integrity

Ensures data remains:

- Accurate
 - Consistent
 - Secure
-

Example

Preventing unauthorized modification of source code.

2. Information Sharing

Allows team members to share project information.

Example

Multiple developers accessing same repository.

3. Tool Integration

Supports integration with development tools.

Example

Integration with Git, Jenkins, testing tools.

4. Data Integration

Combines data from different project activities.

Example

Linking requirements with testing modules.

5. Methodology Enforcement

Ensures software processes are followed properly.

Example

Mandatory code review before merging changes.

6. Document Standardization

Maintains consistent documentation formats.

Example

Standard templates for test reports.

Advantages of SCM Repository

1. Centralized management
 2. Improved collaboration
 3. Better security
 4. Faster recovery from errors
 5. Easy version management
-

Real Life Examples

Popular SCM repositories:

1. GitHub
 2. GitLab
 3. Bitbucket
 4. SVN repositories
-

Importance of SCM Repository

1. Supports distributed development
 2. Prevents data loss
 3. Improves software quality
 4. Enables version control
 5. Helps change tracking
-

Conclusion

SCM Repository is a central component of Software Configuration Management. It stores software artifacts, manages versions, supports collaboration, and ensures consistency and integrity throughout software development.

Q10. Explain Version Control, Change Control, and Configuration Audit.

Answer

Introduction

Software systems continuously evolve because of:

- Requirement changes
- Bug fixes
- Feature enhancements

To manage these changes properly, Software Configuration Management uses:

1. Version Control
2. Change Control
3. Configuration Audit

These activities ensure software consistency and quality.

1. Version Control

Definition

Version Control is the process of managing multiple versions of software artifacts systematically.

According to module notes, version control combines procedures and tools to manage different software versions.

Purpose

1. Maintain software history
 2. Track modifications
 3. Support rollback
 4. Enable team collaboration
-

Features of Version Control

1. Repository management
2. Version tracking
3. Merge support
4. Branching support
5. Bug tracking

Example

Version 1.0 → Version 2.0 → Version 3.0

Popular Version Control Tools

1. Git
 2. SVN
 3. Mercurial
-

Advantages

1. Easy rollback
 2. Better collaboration
 3. Tracks modifications
 4. Reduces code conflicts
-

2. Change Control

Definition

Change Control is the process of evaluating, approving, implementing, and tracking software changes systematically.

Purpose

1. Prevent unauthorized changes
 2. Control software modifications
 3. Maintain software stability
-

Change Control Process

According to module notes, the process includes:

1. Change request generated
2. Developer evaluates request
3. Change control authority approves/rejects
4. Changes implemented

5. Testing performed
6. New version released

Diagram

Change Request

↓

Evaluation

↓

Approval

↓

Implementation

↓

Testing

↓

Release

Example

Customer requests:

- New payment option in e-commerce system.

After approval:

- Developers implement feature
 - Testers validate changes
-

Advantages

1. Better software quality
 2. Controlled modifications
 3. Reduced project risk
 4. Improved traceability
-

3. Configuration Audit

Definition

Configuration Audit is the process of verifying whether software changes are implemented correctly and configuration items are properly maintained.

According to module notes, configuration audit checks:

- Correct implementation of changes
- Whether SCIs are updated properly.

Objectives

1. Verify software consistency
2. Detect unauthorized changes
3. Ensure process compliance
4. Improve software quality

Types of Audits

1. Functional Audit
2. Physical Audit

Example

Audit verifies:

- Updated source code
- Updated documents
- Correct version release

Advantages of Configuration Audit

1. Ensures software integrity
2. Improves reliability
3. Detects configuration errors
4. Supports quality assurance

Difference Between Version Control, Change Control and Configuration Audit

Activity	Purpose
Version Control	Manage software versions

Activity	Purpose
Change Control	Control software changes
Configuration Audit	Verify correctness of changes

Conclusion

Version Control, Change Control, and Configuration Audit are important SCM activities that help manage software evolution systematically. They improve software quality, maintainability, and reliability throughout the software development life cycle.

Q11. Explain the Testing Life Cycle and Principles of Testing.

Answer

Introduction

Testing is one of the most important phases in software engineering. It ensures that the developed software works correctly according to customer requirements.

Software testing helps:

- Detect defects
- Improve software quality
- Increase customer confidence
- Ensure reliability

Testing is performed systematically through a process called:

- Software Testing Life Cycle (STLC)
-

Definition of Software Testing

Software Testing is the process of executing software with the intent of finding defects and verifying that the software meets specified requirements.

Software Testing Life Cycle (STLC)

Definition

STLC is a sequence of activities performed during the software testing process to ensure software quality.

Each phase has:

- Specific objectives
- Inputs

- Outputs

Phases of Testing Life Cycle

1. Requirement Analysis
2. Test Planning
3. Test Case Development
4. Test Environment Setup
5. Test Execution
6. Defect Reporting and Tracking
7. Test Closure

Diagram of STLC

Requirement Analysis

↓

Test Planning

↓

Test Case Development

↓

Test Environment Setup

↓

Test Execution

↓

Defect Reporting

↓

Test Closure

1. Requirement Analysis

Purpose

Understand:

- Functional requirements
- Non-functional requirements

Activities

1. Study SRS document
2. Identify testable requirements
3. Determine testing scope

Output

- Requirement Traceability Matrix (RTM)

Example

Analyzing login module requirements.

2. Test Planning**Purpose**

Prepare complete testing strategy.

Activities

1. Define testing objectives
2. Select testing techniques
3. Allocate resources
4. Estimate cost and schedule

Output

- Test Plan Document

Example

Planning:

- Functional testing
 - Security testing
 - Performance testing
-

3. Test Case Development

Purpose

Create test cases and test data.

Activities

1. Write test cases
 2. Prepare test data
 3. Review test cases
-

Example Test Case

Input	Expected Output
-------	-----------------

Correct password	Login successful
------------------	------------------

Output

- Test cases
 - Test scripts
-

4. Test Environment Setup

Purpose

Prepare hardware and software environment for testing.

Activities

1. Install software
 2. Configure database
 3. Set up network
-

Example

Preparing:

- Web server
- Browser environment
- Database server

Output

- Test environment ready
-

5. Test Execution

Purpose

Execute test cases and identify defects.

Activities

1. Run test cases
 2. Compare actual vs expected results
 3. Record defects
-

Example

Testing payment gateway functionality.

Output

- Test execution reports
-

6. Defect Reporting and Tracking

Purpose

Document and track software defects.

Activities

1. Report bugs
 2. Assign defects to developers
 3. Verify fixes
-

Example

Bug:

- Incorrect balance calculation

Output

- Defect reports
-

7. Test Closure

Purpose

Evaluate testing completion.

Activities

1. Review test coverage
 2. Prepare final reports
 3. Analyze lessons learned
-

Output

- Test summary report
-

Principles of Software Testing

Software testing follows important principles.

1. Testing Shows Presence of Defects

Testing can prove:

- Defects exist

But cannot prove:

- Software is completely error free.
-

Example

Even after testing, hidden bugs may remain.

2. Exhaustive Testing is Impossible

Testing all possible inputs is impossible.

Example

Mobile app with millions of input combinations.

3. Early Testing Saves Cost

Testing should start early in development.

Example

Requirement defects found early cost less to fix.

4. Defect Clustering

Most defects occur in few modules.

Example

Payment module may contain majority of bugs.

5. Pesticide Paradox

Repeating same tests repeatedly becomes ineffective.

Solution

Create new test cases regularly.

6. Testing Depends on Context

Testing method depends on software type.

Example

Banking software requires stronger security testing.

7. Absence of Errors Fallacy

Error-free software is useless if it does not meet user requirements.

Example

Perfectly working software with wrong functionality.

Advantages of Testing

1. Improves software quality
 2. Reduces defects
 3. Enhances reliability
 4. Improves customer satisfaction
 5. Reduces maintenance cost
-

Conclusion

Software Testing Life Cycle provides a systematic approach for testing software effectively. Testing principles help testers design efficient testing strategies and improve software quality and reliability.

Q12. Differentiate between Verification and Validation with suitable examples.

Answer

Introduction

Verification and Validation are important quality assurance activities in software engineering.

Both help improve software quality but differ in:

- Purpose
- Process
- Execution method

Verification checks whether software is built correctly, while validation checks whether the correct software is built.

Verification

Definition

Verification is the process of evaluating software work products to ensure they meet specified requirements.

It answers:

“Are we building the product right?”

Verification focuses on:

- Documents
- Design
- Code reviews

without executing the software.

Characteristics of Verification

1. Static process
 2. No code execution
 3. Performed during development
 4. Prevents defects early
-

Verification Activities

1. Reviews
 2. Inspections
 3. Walkthroughs
 4. Static analysis
-

Example

Reviewing SRS document to ensure all requirements are correct.

Advantages of Verification

1. Detects errors early
 2. Reduces development cost
 3. Improves software quality
 4. Prevents defects propagation
-

Validation

Definition

Validation is the process of evaluating software during or after execution to ensure it satisfies user requirements.

It answers:

“Are we building the right product?”

Validation focuses on:

- Executing software
 - Testing functionality
-

Characteristics of Validation

1. Dynamic process
 2. Requires code execution
 3. Performed after development stages
 4. Detects functional defects
-

Validation Activities

1. Unit testing
 2. Integration testing
 3. System testing
 4. Acceptance testing
-

Example

Testing login functionality using valid and invalid passwords.

Advantages of Validation

1. Ensures correct functionality
 2. Improves customer satisfaction
 3. Detects runtime defects
 4. Confirms requirement fulfillment
-

Difference Between Verification and Validation

Verification

Checks whether product is built correctly

Static process

Validation

Checks whether correct product is built

Dynamic process

Verification

No code execution

Performed during development

Focuses on process

Detects defects early

Examples: reviews, inspections

Validation

Requires code execution

Performed after development

Focuses on product

Detects runtime issues

Examples: testing

Simple Analogy

Verification → Checking blueprint of house

Validation → Checking completed house

Real Life Example

Suppose software requirement:

- ATM should allow cash withdrawal.
-

Verification Example

Checking:

- Design documents
 - Flowcharts
 - Algorithms
-

Validation Example

Actually testing:

- Cash withdrawal operation
-

Importance of Verification and Validation

1. Improve software quality
2. Reduce defects
3. Ensure requirement compliance
4. Increase reliability

5. Improve customer confidence

Diagram

Verification → Prevent defects

Validation → Detect defects

Conclusion

Verification and Validation are essential quality assurance activities in software engineering. Verification ensures software is developed correctly, while validation ensures the developed software satisfies customer requirements. Together they improve software quality and reliability.

Q13. (Application Based) A software company is developing an Online Banking System. Identify possible risks and prepare an RMMM Plan for the project.

Answer

Introduction

An Online Banking System is a critical software application that handles:

- Financial transactions
- Customer accounts
- Online payments
- Security-sensitive operations

Such systems involve many risks related to:

- Security
- Technology
- Performance
- Project management

Therefore, preparing an RMMM Plan is essential.

RMMM stands for:

1. Risk Mitigation
 2. Risk Monitoring
 3. Risk Management
-

Possible Risks in Online Banking System

Major risks include:

1. Security Risks
 2. Technical Risks
 3. Project Risks
 4. Performance Risks
 5. Business Risks
-

1. Security Risks

Examples

1. Hacking attacks
 2. Data theft
 3. Unauthorized access
 4. Phishing attacks
-

Impact

- Financial loss
 - Customer trust issues
-

2. Technical Risks

Examples

1. Server failure
 2. Database crash
 3. Integration failure
 4. Software bugs
-

Impact

- System downtime
 - Incorrect transactions
-

3. Project Risks

Examples

1. Requirement changes
 2. Staff shortage
 3. Schedule delays
-

Impact

- Increased project cost
 - Delayed delivery
-

4. Performance Risks

Examples

1. Slow transaction processing
 2. High server load
-

Impact

- Poor user experience
-

5. Business Risks

Examples

1. Regulatory changes
 2. Customer dissatisfaction
 3. Market competition
-

RMMM Plan for Online Banking System

Risk 1 — Security Breach

Category	Details
----------	---------

Risk	Unauthorized access/hacking
------	-----------------------------

Probability	High
-------------	------

Impact	Catastrophic
--------	--------------

Mitigation

1. Use encryption
 2. Multi-factor authentication
 3. Firewall implementation
 4. Security testing
-

Monitoring

1. Monitor login attempts
 2. Intrusion detection systems
 3. Daily security audits
-

Management

1. Activate disaster recovery plan
 2. Block suspicious accounts
 3. Restore backup data
-

Risk 2 — Server Failure

Category Details

Risk Banking server crash

Probability Medium

Impact Critical

Mitigation

1. Maintain backup servers
 2. Load balancing
 3. Regular maintenance
-

Monitoring

1. CPU monitoring

2. Server health checks
 3. Network monitoring
-

Management

1. Switch to backup server
 2. Notify users
 3. Recover failed services
-

Risk 3 — Requirement Changes

Category Details

Risk Frequent customer requirement changes

Probability Medium

Impact Medium

Mitigation

1. Requirement validation meetings
 2. Proper documentation
 3. Change control process
-

Monitoring

1. Track change requests
 2. Conduct weekly reviews
-

Management

1. Update schedules
 2. Allocate additional resources
-

Risk 4 — Staff Turnover

Category Details

Risk Key developer leaving project

Category Details

Probability Medium

Impact High

Mitigation

1. Cross-training employees
 2. Maintain documentation
-

Monitoring

1. Employee satisfaction reviews
 2. Team meetings
-

Management

1. Hire replacement staff
 2. Reassign project tasks
-

Diagram of RMMM Process

Risk Identification

↓

Risk Mitigation

↓

Risk Monitoring

↓

Risk Management

Advantages of RMMM Plan

1. Better project control
2. Reduced project failure
3. Improved security
4. Better resource management

5. Increased customer trust

Conclusion

An Online Banking System faces several technical, security, and project risks. A proper RMMM Plan helps identify, monitor, and manage these risks effectively, ensuring secure and successful project completion.

Q14. (Application Based) Prepare a Test Plan for an E-Commerce Website, mentioning different phases and types of testing to be performed.

Answer

Introduction

An E-Commerce Website allows users to:

- Browse products
- Add items to cart
- Make online payments
- Track orders

Since it handles:

- Customer information
- Financial transactions
- Real-time operations

proper testing is extremely important.

A Test Plan is prepared to define:

- Testing objectives
- Testing strategy
- Scope
- Resources
- Schedule
- Testing activities

Definition of Test Plan

A Test Plan is a formal document that describes the scope, objectives, strategy, schedule, and resources required for software testing.

Objectives of Test Plan

1. Ensure software quality
 2. Detect defects
 3. Verify requirements
 4. Improve reliability
 5. Ensure secure transactions
-

Scope of Testing

The E-Commerce Website includes modules such as:

1. User Registration
 2. Login System
 3. Product Search
 4. Shopping Cart
 5. Payment Gateway
 6. Order Tracking
 7. Admin Panel
-

Test Plan Structure

1. Test Plan Identifier

E-Commerce Website Test Plan

Version 1.0

2. Testing Objectives

1. Validate website functionality
 2. Verify payment security
 3. Ensure performance under load
 4. Check compatibility across devices
-

3. Testing Phases

The testing process will include several phases.

Phase 1 — Requirement Analysis

Activities

1. Study SRS document
2. Identify testable requirements
3. Understand business flow

Example

Analyze:

- Login functionality
- Payment workflow

Output

- Requirement Traceability Matrix (RTM)

Phase 2 — Test Planning

Activities

1. Define testing strategy
2. Allocate testing resources
3. Estimate schedule and cost

Output

- Test Plan Document

Phase 3 — Test Case Design

Activities

1. Prepare test cases
2. Create test data
3. Review test cases

Example Test Case

Test Case Expected Result

Valid login User dashboard displayed

Output

- Test cases
 - Test scripts
-

Phase 4 — Test Environment Setup

Activities

1. Configure servers
 2. Set up browsers
 3. Prepare database
-

Environment Includes

1. Chrome browser
 2. Firefox browser
 3. Mobile devices
 4. Payment gateway sandbox
-

Phase 5 — Test Execution

Activities

1. Execute test cases
 2. Record results
 3. Identify defects
-

Example

Testing:

- Add to cart functionality
-

Output

- Test execution reports

Phase 6 — Defect Reporting and Tracking

Activities

1. Report bugs
2. Assign bugs to developers
3. Retest fixed defects

Example Defect

Payment successful but order not generated

Phase 7 — Test Closure

Activities

1. Prepare final report
 2. Analyze testing results
 3. Document lessons learned
-

Types of Testing to be Performed

1. Functional Testing

Purpose

Verify website functions correctly.

Modules Tested

1. Login
 2. Registration
 3. Cart
 4. Checkout
 5. Payment
-

Example

Verify:

- User can add products to cart.
-

2. Unit Testing

Purpose

Test individual modules separately.

Example

Testing:

- Product search function
-

3. Integration Testing

Purpose

Check interaction between modules.

Example

Verify:

- Payment gateway integration with order module
-

4. System Testing

Purpose

Test complete integrated website.

Example

Verify complete shopping process.

5. User Acceptance Testing (UAT)

Purpose

Validate software according to customer requirements.

Example

Customer checks:

- Real-world shopping workflow
-

6. Performance Testing

Purpose

Check website performance under heavy load.

Example

Testing:

- 10,000 concurrent users
-

7. Security Testing

Purpose

Identify security vulnerabilities.

Example

Testing:

- SQL Injection
 - Password encryption
 - Session management
-

8. Compatibility Testing

Purpose

Check website across different devices and browsers.

Example

Testing on:

1. Chrome
 2. Firefox
 3. Edge
 4. Mobile devices
-

9. Usability Testing

Purpose

Evaluate user friendliness.

Example

Check:

- Navigation simplicity
 - Checkout ease
-

10. Regression Testing

Purpose

Ensure new changes do not affect existing functionality.

Example

After payment update:

- Verify cart still works correctly.
-

Testing Tools

Testing Type	Tools
Automation Testing	Selenium
Performance Testing	JMeter
Bug Tracking	Jira
Security Testing	OWASP ZAP

Risks in Testing

1. Payment failures
 2. Security vulnerabilities
 3. Slow server response
 4. Browser compatibility issues
-

Deliverables

1. Test Plan
 2. Test Cases
 3. Bug Reports
 4. Test Summary Report
 5. RTM Document
-

Advantages of Proper Test Plan

1. Better software quality
 2. Reduced defects
 3. Improved security
 4. Better customer satisfaction
 5. Faster defect resolution
-

Conclusion

A proper Test Plan ensures systematic testing of the E-Commerce Website. By performing functional, security, performance, compatibility, and usability testing, the organization can deliver reliable, secure, and high-quality software to customers.

Q15. (Application Based) A team is maintaining multiple versions of a College ERP System. Suggest a suitable SCM process including version control and change management.

Answer

Introduction

A College ERP System manages:

- Student records
- Attendance
- Fees
- Exams
- Library management
- Staff information

As the ERP system evolves:

- New features are added

- Bugs are fixed
- Requirements change

Managing multiple versions without proper control becomes difficult.

Therefore, an effective Software Configuration Management (SCM) process is required.

Definition of SCM

Software Configuration Management (SCM) is the process of identifying, controlling, tracking, and managing changes made to software throughout its life cycle.

Objectives of SCM for ERP System

1. Manage multiple software versions
 2. Track software changes
 3. Improve collaboration
 4. Prevent code conflicts
 5. Maintain software consistency
-

Suitable SCM Process for College ERP System

The SCM process should include:

1. Configuration Identification
 2. Version Control
 3. Change Control
 4. Configuration Status Accounting
 5. Configuration Audit
 6. Build and Release Management
-

SCM Process Diagram

Configuration Identification



Version Control



Change Control



Build & Release



Configuration Audit

1. Configuration Identification

Purpose

Identify all Software Configuration Items (SCIs).

SCIs in ERP System

1. Source code
 2. Database scripts
 3. Test cases
 4. User manuals
 5. Design documents
-

Example

ERP Version 2.0

Attendance Module

Fee Module

Library Module

Advantages

1. Better organization
 2. Easier tracking
 3. Improved maintainability
-

2. Version Control

Purpose

Manage multiple versions of ERP software systematically.

Suggested Tool

Git or GitHub

Versioning Strategy

Version 1.0 → Initial Release

Version 1.1 → Bug Fixes

Version 2.0 → New Features

Branching Strategy

Branch	Purpose
Main Branch	Stable production version
Development Branch	Ongoing development
Feature Branch	New feature development
Bug Fix Branch	Error correction

Example

feature/student-attendance

bugfix/fee-calculation

Advantages of Version Control

1. Easy rollback
 2. Better collaboration
 3. Tracks changes
 4. Prevents overwriting code
-

3. Change Control Process

Purpose

Manage software modifications systematically.

Change Control Steps

Step 1 — Change Request

User submits request.

Example

College requests:

- Online examination module
-

Step 2 — Impact Analysis

Analyze:

- Cost
 - Schedule
 - Dependencies
-

Step 3 — Approval

Change Control Board (CCB) approves or rejects request.

Step 4 — Implementation

Developers implement approved changes.

Step 5 — Testing

Testing team validates modifications.

Step 6 — Release

New ERP version deployed.

Change Control Workflow

Change Request

↓

Impact Analysis

↓

Approval



Development



Testing



Release

4. Configuration Status Accounting

Purpose

Maintain records of:

- Versions
 - Changes
 - Releases
-

Example

Tracking:

- Which module updated
 - Who modified code
 - Release dates
-

5. Configuration Audit

Purpose

Verify:

- Correct implementation of changes
 - Updated documentation
 - Proper version release
-

Types of Audit

1. Functional Audit
2. Physical Audit

Example

Verify:

- Attendance module updated correctly
-

6. Build and Release Management

Purpose

Generate stable software releases.

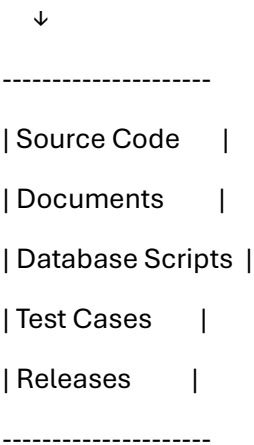
Example Release Strategy

Version Purpose

- v1.0 Initial deployment
 - v1.1 Minor bug fixes
 - v2.0 Major feature upgrade
-

Repository Structure

ERP Repository



Recommended SCM Tools

Activity	Tool
Version Control	Git

Activity	Tool
Repository Hosting	GitHub/GitLab
Bug Tracking	Jira
CI/CD	Jenkins

Advantages of SCM Process

1. Better version management
 2. Easier collaboration
 3. Improved software quality
 4. Controlled software changes
 5. Faster recovery from errors
-

Real Life Scenario

Suppose:

- ERP v2.0 has fee calculation bug

Using SCM:

- Developers rollback to stable version
 - Fix issue
 - Release patched version safely
-

Conclusion

A proper SCM process is essential for maintaining multiple versions of a College ERP System. Version control, change management, configuration audit, and release management help maintain software quality, consistency, and reliability throughout the software life cycle.